

PESC/COPPE/UFRJ · PESQUISA DE
DOUTORADO

Endo-DSL: Projetando Jogos Educativos com Linguagens de Domínio Específico e Taxonomia de Bloom

Da formalização de construtos de game design educacional à síntese assistida por pipeline multi-agente e à avaliação comparativa automático × manual.



Autor: Caio Sazeredo

Programa: Engenharia de Sistemas e Computação (PESC)

Instituição: COPPE/UFRJ — Rio de Janeiro, Brasil

Contato: caiosazeredo@cos.ufrj.br

Data: Junho de 2026

Versão: 1.1.0

FICHA CATALOGRÁFICA

Sazeredo, Caio

Endo-DSL: Projetando Jogos Educativos com Linguagens de Domínio Específico e Taxonomia de Bloom / Caio Sazeredo. — Rio de Janeiro: PESC/COPPE/UFRJ, 2026.

Inclui referências bibliográficas e apêndices.

1. Linguagens de domínio específico. 2. Taxonomia de Bloom. 3. Jogos educativos endógenos. 4. Pipeline multi-agente. 5. Compilação HTML5. 6. Design Science Research. I. Título.

CDD 005.13 · CDU 004.43:371.3

"Ensinar não é transferir conhecimento, mas criar possibilidades para a sua produção ou a sua construção." — Paulo Freire

Este livro narra a construção de uma linguagem que tenta tornar verificável o que foi ensinado, o que foi jogado, e — esperamos — o que foi aprendido.

ÍNDICE

1. Introdução: Contexto, Motivação e Objetivos 1

O problema dos jogos exógenos

A proposta Endo-DSL

Como usar este livro

2. Fundamentos de DSLs 2

O que é uma DSL?

Por que DSL para jogos educativos?

Exemplos de DSLs educacionais

3. Taxonomia Revisada de Bloom 3

Os seis níveis cognitivos

Verbos de Bloom e objetivos de aprendizagem

Bloom no design de jogos

4. A Linguagem Endo-DSL 4

Gramática EBNF anotada

Construtos: game, metadata, objective, mechanic,
loop, narrative

Exemplos e erros comuns

5. Biblioteca de Componentes 5

O que são componentes?

Curadoria e ciclo de vida

Como contribuir

6. Pipeline Multi-Agente 6

As fases de recuperação, geração e validação

LLM offline vs. Claude

Métricas do pipeline

7. **Compilação HTML5** **7**

Arquitetura do compilador

Reparametrização de domínio

Artefatos gerados

8. **Avaliação Pedagógica** **8**

Instrumento Likert 7 dimensões

Relatórios comparativos

Interpretação dos resultados

9. **Estudo de Caso: "Comparando Frações"** **9**

A especificação DSL

O protótipo gerado

A avaliação

10. **Apêndice A: Gramática EBNF Completa**

11. **Apêndice B: Referência de Comandos CLI**

12. **Apêndice C: Glossário**

13. **Apêndice D: Referências**

Introdução: Contexto, Motivação e Objetivos

Jogos educativos de qualidade são raros, caros de produzir e frequentemente mal-executados. Este capítulo apresenta o problema que motiva a Endo-DSL e os objetivos do projeto.

O problema dos jogos exógenos

Imagine um jogo de matemática em que o personagem principal, um robô espacial, precisa responder corretamente perguntas de multiplicação para "recarregar energia" e avançar no mapa. O conteúdo matemático está *fora* do jogo — é um pedágio para continuar jogando. Remova as perguntas e o jogo de plataforma continua funcionando perfeitamente. Isso é um jogo **exógeno**: o conteúdo educativo é um apêndice, não o núcleo.

Agora imagine um jogo em que o personagem só pode atravessar pontes se o jogador construir corretamente as frações que representam o peso máximo suportado. Remova o conhecimento matemático e o jogo deixa de existir. Isso é um jogo **endógeno**: aprender *é* jogar.

A distinção é fundamental. A literatura empírica (HABGOOD; AINSWORTH, 2011; MALONE, 1981) mostra que jogos endógenos produzem resultados de aprendizagem superiores porque motivam intrinsecamente — o aluno resolve o problema curricular porque *quer vencer o jogo*, não porque uma tela intermediária pergunta antes de deixar avançar.

Contudo, projetar um bom jogo endógeno é difícil. Requer que o designer simultaneamente: (1) conheça a mecânica de jogo e suas propriedades motivacionais; (2) domine o conteúdo curricular e a taxonomia cognitiva; e (3) saiba codificá-lo em software. Essa tríade de competências é rara e cara, o que explica por que a maioria dos "jogos educativos" disponíveis são, na prática, quizzes com efeitos especiais.

A proposta Endo-DSL

A **Endo-DSL** é uma resposta a esse problema. É uma linguagem de domínio específico — uma linguagem de computação projetada para um domínio específico — que permite a qualquer designer (com ou sem experiência de programação) especificar formalmente:

- Os **objetivos pedagógicos** do jogo, com seu nível na Taxonomia de Bloom;
- As **mecânicas endógenas** que os exercitam, com afinidade cognitiva verificável;
- Os **loops de jogabilidade** que estruturam o fluxo do jogo;
- As **ramificações narrativas** que contextualizam as escolhas do jogador.

A partir dessa especificação, a plataforma:

- **Valida** a coerência pedagógica (Bloom×mecânica) automaticamente;
- **Compila** um protótipo HTML5 jogável e autocontido;
- **Gera** automaticamente a especificação DSL a partir de um contexto educacional (objetivo + domínio + Bloom + faixa etária), usando recuperação de componentes e inteligência artificial;
- **Avalia** os protótipos em sete dimensões pedagógicas.

O resultado é uma plataforma de pesquisa que torna o design educacional de qualidade mais acessível — sem eliminar o julgamento

humano nos aspectos que requerem sensibilidade contextual e criatividade.

Princípio central: A Endo-DSL formaliza o que é formalizável (tipos de mecânica, objetivos, loops, Bloom), preserva o que é humano (engajamento, tom, estética, adequação cultural), e torna ambos explícitos e auditáveis.

Como usar este livro

Este livro é destinado a pesquisadores, professores e designers instrucionais que querem entender, usar e contribuir com a plataforma Endo-DSL. Pode ser lido sequencialmente — dos fundamentos ao estudo de caso — ou consultado como referência por capítulo.

O Capítulo 4 (a linguagem) e o Capítulo 9 (estudo de caso) são os mais práticos para quem quer começar a escrever especificações DSL imediatamente. Os Capítulos 2 e 3 fornecem o embasamento teórico para pesquisadores e professores que querem entender *por que* as decisões de design foram tomadas.

O código-fonte completo da plataforma está disponível em:

<https://github.com/caiosazeredo/Endo-DSL>

Para iniciar a plataforma com um único comando:

```
pip install -e .
endo-dsl init-db
endo-dsl seed-components
endo-dsl serve --port 8000
# Acesse http://localhost:8000
```


Fundamentos de Linguagens de Domínio Específico

Uma DSL é uma linguagem de computação que troca generalidade por poder expressivo em um domínio específico. Neste capítulo, explicamos o que isso significa e por que faz sentido para o design de jogos educativos.

O que é uma DSL?

Linguagens de programação de propósito geral — como Python, Java ou C++ — podem expressar qualquer algoritmo computável. Sua generalidade, porém, tem um custo: para um professor de matemática, escrever um quiz em Python requer aprender variáveis, loops, funções, bibliotecas de UI. A distância entre o problema (um quiz sobre frações) e a solução (código Python) é enorme.

Uma **linguagem de domínio específico** (DSL — Domain-Specific Language) reduz essa distância ao ser projetada especificamente para um domínio. SQL é uma DSL para consultas a bancos de dados. HTML é uma DSL para documentos web. LaTeX é uma DSL para tipografia acadêmica. A Endo-DSL é uma DSL para design de jogos educativos endógenos estruturados pela Taxonomia de Bloom.

As principais vantagens de uma DSL são:

Vantagem	Descrição	No contexto Endo-DSL
Expressividade	O designer pensa em termos do problema, não da implementação	Escreve <code>bloom:</code> <code>Analisar</code> , não código JavaScript
Verificabilidade	A gramática define o que é correto; erros são detectados antes da execução	Tipo de mecânica inválido → erro no editor, não em runtime
Geração de código	A DSL serve de modelo de alto nível para compiladores gerarem artefatos	DSL → protótipo HTML5 autocontido com um comando
Comunicação	A DSL é uma linguagem compartilhada entre designers e desenvolvedores	Professor e engenheiro de software leem a mesma especificação

DSLs são classificadas em **externas** (com sintaxe própria e parser dedicado, como SQL e LaTeX) e **internas** (embutidas em uma linguagem hospedeira, como as DSLs Ruby do Rails). A Endo-DSL é uma DSL **externa textual**: tem sua própria gramática EBNF, parser recursivo descendente e compilador independente.

Por que DSL para jogos educativos?

Professores e designers instrucionais têm expertise em pedagogia e conteúdo curricular, não em programação de jogos. Ferramentas como Unity, Godot ou GameMaker exigem habilidades técnicas que levam anos para desenvolver. Por outro lado, editores de PowerPoint ou Google Forms permitem criar quizzes mas não jogos com mecânicas integradas ao conteúdo.

Uma DSL para jogos educativos endógenos preenche esse gap: abstrai os detalhes técnicos (HTML, CSS, JavaScript, estruturas de dados) e expõe apenas os conceitos relevantes para o design pedagógico — objetivos, mecânicas, loops, Bloom. O compilador cuida do resto.

Adicionalmente, a formalidade da DSL permite:

- **Verificação automática** de coerência pedagógica — algo impossível com descrições textuais em Word ou PDF;
- **Geração assistida por IA** — o pipeline multi-agente pode gerar e validar especificações porque a DSL define precisamente o que é válido;
- **Reúso sistemático** — componentes pedagógicos em uma biblioteca são reutilizáveis porque têm interface formal (assinatura DSL);
- **Rastreabilidade** — cada elemento do protótipo pode ser rastreado à decisão de design que o originou.

Exemplos de DSLs educacionais

O campo de DSLs para educação é relativamente jovem mas ativo. Alguns exemplos:

- **Scratch** (MIT): DSL visual por blocos para programação de histórias e jogos simples. Altamente acessível mas sem semântica pedagógica formal.

- **Alice** (Carnegie Mellon): ambiente de programação orientado a objetos com narrativa 3D. Educação em CS, sem suporte a Bloom.
- **NetLogo** (Northwestern): DSL para modelagem de sistemas complexos. Forte em Ciências, sem mecânicas de jogo.
- **GameDSL-BR**: proposta brasileira de DSL para jogos educativos em português, sem pipeline de geração automática ou instrumento de avaliação.
- **Endo-DSL** (esta proposta): DSL textual com Bloom como construto de primeira classe, pipeline multi-agente, compilador HTML5 e instrumento de avaliação.

A lacuna que a Endo-DSL preenche é específica: nenhuma das DSLs existentes combina (1) Bloom verificável na sintaxe, (2) geração automática por LLM+RAG, (3) compilação para HTML5 autocontido, e (4) instrumento de avaliação comparativo.

A Taxonomia Revisada de Bloom

A Taxonomia de Bloom é o mapa cognitivo da aprendizagem. Neste capítulo, explicamos seus seis níveis, seus verbos, e como ela se torna um construto de primeira classe na Endo-DSL.

Os seis níveis cognitivos

Originalmente proposta por Bloom et al. (1956) e revisada por Anderson & Krathwohl (2001), a Taxonomia Revisada de Bloom organiza os processos cognitivos em seis níveis de complexidade crescente, do mais simples ao mais elaborado:

1

Lembrar

Recuperar conhecimento da memória de longo prazo. Verbos: identificar, reconhecer, listar, nomear, recordar, definir.

2

Compreender

Construir significado a partir de mensagens. Verbos: explicar, classificar, resumir, interpretar, comparar, exemplificar.

3

Aplicar

Usar procedimentos em situações novas. Verbos: usar, resolver, demonstrar, executar, implementar, calcular.

4

Analisar

Decompor em partes e entender as relações. Verbos: comparar, diferenciar, organizar, inferir, distinguir, analisar.

5

Avaliar

Emitir julgamentos baseados em critérios. Verbos: julgar, criticar, justificar, argumentar, defender, avaliar.

6

Criar

Combinar elementos para formar um novo todo. Verbos: construir, planejar, produzir, compor, projetar, formular.

Os níveis possuem uma **ordem total**: habilidades do nível 4 (Analisar) pressupõem as dos níveis 1–3. Um aluno não pode analisar o que não compreende; não pode avaliar o que não consegue analisar. Essa ordem é explorada pela Endo-DSL para validar a coerência cognitiva entre diferentes elementos da especificação.

Verbos de Bloom e objetivos de aprendizagem

A escolha do verbo no objetivo de aprendizagem determina o nível de Bloom e, conseqüentemente, o tipo de mecânica de jogo adequada. Esta tabela resume as associações mais comuns:

Nível	Verbos típicos	Exemplo de objetivo
Lembrar	identificar, listar, nomear, reconhecer	"Identificar as capitais dos estados brasileiros"
Compreender	explicar, classificar, resumir, interpretar	"Classificar animais em vertebrados e invertebrados"
Aplicar	usar, resolver, demonstrar, calcular	"Resolver problemas de regra de três simples"
Analisar	comparar, diferenciar, organizar, inferir	"Comparar frações com denominadores diferentes"
Avaliar	julgar, criticar, justificar, argumentar	"Avaliar a validade de um argumento científico"
Criar	construir, planejar,	"Construir um modelo matemático"

produzir,
compor

para um problema
real"

Bloom no design de jogos

Cada nível de Bloom tem afinidade natural com certos *tipos de mecânica de jogo*. Um quiz de múltipla escolha é adequado para Lembrar e Compreender — o jogador recupera ou interpreta informações. Uma atividade de classificação é afim a Analisar — o jogador decompõe e organiza. Um puzzle aberto é afim a Criar — o jogador constrói algo novo.

A Endo-DSL codifica essas afinidades como regras semânticas verificáveis. Quando um designer declara uma mecânica do tipo `quiz` com `bloom: Criar`, o validador emite um aviso: quiz não tem afinidade cognitiva com Criar. Isso não impede a compilação — o designer pode ter uma boa razão para a escolha — mas torna a divergência visível e auditável.

Na Endo-DSL: os seis níveis de Bloom são um *não-terminal* da gramática formal. Não são metadados textuais livres — são verificados sintaticamente pelo parser e semanticamente pelo validador.

A Linguagem Endo-DSL

Este capítulo é um guia de referência prática da linguagem: gramática, construtos, exemplos anotados e erros comuns.

Gramática EBNF — visão geral

A gramática da Endo-DSL é escrita em EBNF (Extended Backus-Naur Form, ISO/IEC 14977), a notação padrão para gramáticas de linguagens formais. Os principais operadores são: `{ x }` = zero ou mais ocorrências; `[ x ]` = opcional; `|` = alternativa; terminais entre aspas duplas.

O ponto de entrada é `spec = game_decl`: toda especificação é uma declaração de jogo. A estrutura de alto nível:

```
(* ponto de entrada *)
spec      = game_decl ;
game_decl = "game" , string , "{" , {
game_member } , "}" ;
game_member = metadata_block
              | objective_decl
              | mechanic_decl
              | loop_decl
              | narrative_decl ;

(* Bloom como construto de primeira classe
- RF02 *)
bloom = "Lembrar" | "Compreender" |
        "Aplicar"
```

```
| "Analisar" | "Avaliar" |  
"Criar" ;
```

A lista completa dos não-terminais está no Apêndice A. O destaque aqui é o não-terminal `bloom`: ele aceita exatamente seis valores (em português ou inglês), e qualquer outro valor é um erro de sintaxe detectado pelo parser.

Construtos em detalhe

game — a declaração raiz

Toda especificação começa com `game` seguido de um título entre aspas e um bloco entre chaves:

```
game "Comparando Frações" {  
  // membros aqui  
}
```

metadata — contexto educacional

```
metadata {  
  domain:    "Matemática"  
  topic:     "frações"  
  bloom:     Analisar  
  age_range: "10-11"  
  duration:  15  
}
```

O bloco `metadata` armazena o contexto pedagógico. O campo `bloom` aqui declara o nível dominante do jogo como um todo — usado pelo pipeline RAG para recuperar componentes afins.

objective — objetivo pedagógico

```
objective obj_comparar {  
  description: "Comparar frações com  
denominadores diferentes usando  
equivalência"  
  bloom:      Analisar  
}
```

Cada objetivo pedagógico é declarado com um identificador único, uma descrição em linguagem natural e o nível de Bloom. Um objetivo deve ser endereçado por pelo menos uma mecânica via `addresses`.

mechanic — mecânica de jogo endógena

```
mechanic comparacao {  
  type:      classification    // enum  
            fechado de tipos  
  bloom:      Analisar          // afim a  
            classification ✓  
  addresses:  obj_comparar       // id do  
            objective  
  description: "O jogador classifica pares  
de frações em maior, menor ou igual."  
  params {  
    items:    6  
    feedback: immediate  
  }  
}
```

O campo `type` aceita um enum fechado de tipos de mecânica. Os tipos disponíveis são: `quiz`, `classification`, `comparison`, `matching`, `sequencing`, `sorting`, `puzzle`, `narrative_branch`. Cada tipo tem afinidade cognitiva com certos níveis de Bloom — verificada pelo validador semântico.

loop — loop de jogabilidade

```
loop principal {
  bloom:      Analisar
  description: "Classificar e revisar até
consolidar a comparação."
  steps {
    comparacao -> revisao
    revisao    -> comparacao
  }
}
```

O bloco `steps` define o grafo de transições do jogo. Cada linha é uma transição: `mecânica_origem -> mecânica_destino`. O compilador verifica que os identificadores nas transições correspondem a mecânicas declaradas.

narrative — ramificação narrativa

```
narrative historia {
  branch introducao {
    bloom: Compreender
    text:
      "1/2 e 2/4 são equivalentes. Por quê?"
    choice "Representam a mesma parte" ->
correto
    choice "São números diferentes" ->
revisao
  }
  branch correto {
    bloom: Avaliar
    text: "Exato! Frações equivalentes
representam a mesma quantidade."
  }
  branch revisao {
    bloom: Lembrar

```

```
text: "Vamos rever: multiplique
numerador e denominador pelo mesmo número."
}
}
```

Erros comuns e como corrigir

Erro	Causa	Solução
<code>tipo</code> <code>desconhecido</code> <code>'adventure'</code>	Tipo de mecânica fora do enum	Use um dos 8 tipos válidos
<code>bloom inválido</code> <code>'analisar'</code>	Minúscula no início	Capitalize: <code>Analisar</code>
<code>alvo 'obj_x'</code> <code>não declarado</code>	<code>addresses</code> aponta para objetivo objetivo inexistente	Declare o objetivo ou corrija o id
Aviso: mecânica abaixo do nível-alvo	Mecânica com bloom menor que o objetivo	Válido, mas verifique se é intencional
<code>string não fechada</code>	Aspas não balanceadas	Feche a string com <code>"</code>

Biblioteca de Componentes Pedagógicos

A biblioteca é o repositório de conhecimento da plataforma. Componentes bem curados são o que torna a geração automática de qualidade possível.

O que são componentes?

Um **componente pedagógico** é uma unidade reutilizável de design de jogo educativo. Cada componente contém:

- **Assinatura DSL:** o esqueleto de código DSL que define a mecânica;
- **Metadados pedagógicos:** nível de Bloom, tipo de mecânica, domínio, faixa etária sugerida, contexto educacional;
- **Parâmetros configuráveis:** os aspectos ajustáveis (número de questões, dificuldade, feedback);
- **Referências bibliográficas:** a base pedagógica que fundamenta a escolha da mecânica;
- **Métricas de uso:** número de instâncias em protótipos compilados, avaliação média, taxa de sucesso de compilação.

Os componentes são classificados em dois status:

Status	Descrição	Prioridade no RAG
--------	-----------	-------------------

★ Canônico	Aprovado pelo curador: afinidade Bloom×mecânica verificada, documentado, testado	Alta — priorizados na recuperação automática
○ Experimental	Submetido pela comunidade, aguardando curadoria	Normal — usados se não houver canônicos suficientes

Curadoria e ciclo de vida

O ciclo de vida de um componente tem quatro estados: **experimental** (recém-submetido) → **em revisão** (curador alocado) → **canônico** (aprovado) ou **rejeitado** (inadequado). O processo de curadoria registra obrigatoriamente o curador e a justificativa, formando uma trilha auditável.

Os critérios de aprovação recomendados são:

1. O tipo de mecânica tem afinidade cognitiva com o nível de Bloom declarado;
2. A endogeneidade está explicitada — o conteúdo é integral, não decorativo;
3. Os parâmetros são documentados e têm defaults razoáveis;
4. Há pelo menos uma referência bibliográfica ou uma instância compilada e avaliada.

Como contribuir com novos componentes

Qualquer pesquisador ou desenvolvedor pode submeter componentes experimentais via:

- A API HTTP: `POST /api/contribute` com os campos obrigatórios;
- A fachada Python:
`platform.contribute_component(**campos)`;
- Importação em lote: `endo-dsl import --in componentes.json`.

Após a submissão, o componente entra na fila de curadoria visível em `/curator` na interface web ou via `endo-dsl curate list` na CLI.

Dica de pesquisa: Componentes com muitas instâncias (campo `instantiations`) e avaliação média ≥ 4.0 são os melhores candidatos a canônico — indicam que o componente foi testado em contextos reais com boa qualidade percebida.

Pipeline Multi-Agente

O pipeline é o coração inteligente da plataforma — a sequência de agentes que transforma um contexto educacional de alto nível em uma especificação DSL válida.

As fases de recuperação, geração e validação

O pipeline multi-agente opera em quatro fases encadeadas:

Fase	Agente	Entrada	Saída
1 — Contexto	ContextAgent	Formulário (domínio, objetivo, Bloom, faixa etária)	DesignContext estruturado
2 — Recuperação	RetrievalAgent (RAG)	DesignContext	Lista de componentes ranqueados por afinidade
3 — Geração	GenerationAgent (LLM)	DesignContext + componentes recuperados	Especificação DSL candidata

4 —	ValidationAgent	DSL candidata	DSL válida
Validação +	+ loop		(ou falha após
Refinamento			3 tentativas)

A fase de validação é o diferencial: o agente não simplesmente aceita ou rejeita a geração — ele *retorna os erros como feedback estruturado* para o gerador tentar novamente. Este padrão "Self-Refine" garante que o pipeline sempre entrega uma DSL sintaticamente e semanticamente válida, ou informa precisamente por que falhou.

Backend heurístico vs. Claude

O pipeline suporta dois backends de geração intercambiáveis:

Backend	Descrição	Quando usar
template	Gerador heurístico determinístico baseado em templates parametrizados. Sem rede, sem API, zero custo variável.	Experimentos reprodutíveis, CI/CD, demos offline, pesquisa controlada
claude	Claude (Anthropic) via API. Requer <code>ANTHROPIC_API_KEY</code> . Produz DSL mais contextualizada e criativa.	Geração de alta qualidade para uso em produção ou protótipos avançados

O backend `template` é fundamental para a **reprodutibilidade científica**: os mesmos experimentos podem ser replicados anos depois sem depender da disponibilidade ou comportamento de APIs exter-

nas. A tese usa o backend heurístico como linha de base para todos os experimentos de geração.

Ativação automática: Se `ANTHROPIC_API_KEY` estiver definido e `ENDO_DSL_BACKEND` não for explicitamente `template`, o sistema promove automaticamente para o backend `claude`.

Métricas do pipeline

O pipeline registra métricas a cada execução:

- **Número de tentativas:** quantas gerações foram necessárias (ideal: 1);
- **Taxa de sucesso:** % de contextos que geram DSL válida em ≤ 3 tentativas;
- **Componentes usados:** quais componentes da biblioteca foram instanciados;
- **Backend:** qual backend foi usado.

Acesse as métricas via `endo-dsl --json stats` ou pela seção "Métricas de geração" na sidebar do Studio após uma geração.

Compilação para HTML5

O compilador transforma especificações DSL em protótipos HTML5 autocontidos e jogáveis. Neste capítulo, explicamos a arquitetura, os artefatos gerados e a reparametrização.

Arquitetura do compilador

O compilador DSL → HTML5 opera em quatro estágios:

1. **Parse:** o parser recursivo descendente transforma o texto DSL em uma AST (Árvore Sintática Abstrata) tipada;
2. **Validação semântica:** o validador verifica coerência entre bloom dos objetivos e bloom das mecânicas, referências, tipos;
3. **Content pack:** o módulo `content.py` constrói o JSON com os dados do jogo (mecânicas, parâmetros, Bloom, domínio);
4. **Renderização:** o `engine.py` combina o content pack com templates HTML/CSS/JavaScript para produzir o HTML5 autocontido.

O HTML5 gerado é completamente autocontido: CSS, JavaScript, imagens e dados estão embutidos no arquivo único. O usuário pode copiá-lo para um pendrive, enviá-lo por e-mail ou incorporá-lo em um e-book e ele funcionará offline em qualquer navegador moderno.

Reparametrização de domínio (RF22)

Um dos recursos mais poderosos do compilador é a **reparametrização de conteúdo**: trocar o domínio de um protótipo já compilado sem recompilar a estrutura.

Mecanicamente, o compilador emite um bloco especial no HTML:

```
<script id="endo-content"
type="application/json">
  { "domain": "Matemática", "stages":
  [...] }
</script>
```

A reparametrização extrai e reescreve apenas esse bloco JSON com conteúdo de outro domínio, preservando todo o JavaScript do jogo intocado. O resultado é um novo `prototype.reparam.html` que usa a mesma estrutura de mecânica mas com conteúdo de Ciências, História ou qualquer outro domínio.

```
endo-dsl reparametrize 17 --domain
"Ciências" --topic "classificação de
rochas"
# ✓ Protótipo #17 reparametrizado para
'Ciências'.
```

Artefatos gerados

Cada compilação gera quatro artefatos no diretório `<workspace>/proto_<id>/`:

Arquivo	Descrição
<code>prototype.html</code>	

	Jogo HTML5 autocontido e jogável
<code>prototype.traceability.html</code>	Documento de rastreabilidade pedagógica legível
<code>prototype.traceability.json</code>	Rastreabilidade em JSON estruturado (para análise)
<code>prototype.content.json</code>	Content pack para reparametrização

O documento de rastreabilidade (RF23) mapeia, para cada objetivo pedagógico: quais mecânicas o endereçam, em qual nível de Bloom, e quais componentes da biblioteca foram usados. É o "documento científico" do protótipo.

Avaliação Pedagógica

A avaliação transforma a intuição sobre qualidade em dados mensuráveis e comparáveis. O instrumento `endo-eval-1.0` é o fechamento do ciclo científico da plataforma.

O instrumento Likert de 7 dimensões

O instrumento `endo-eval-1.0` avalia cada protótipo em sete dimensões em escala Likert de 1 a 5, com pesos de importância:

#	Dimensão	O que mede	Peso
1	Alinhamento pedagógico	O jogo atende ao objetivo declarado?	1.3
2	Coerência cognitiva (Bloom)	As mecânicas exercitam o nível cognitivo certo?	1.2
3	Endogeneidade	O conteúdo é integrado à mecânica ou externo?	1.3
4	Clareza instrucional	As instruções são compreensíveis para o público?	1.0

5	Adequação ao público	O jogo é adequado à faixa etária/escolaridade?	1.0
6	Potencial de engajamento	O jogo motiva a continuar jogando?	1.0
7	Adaptabilidade de conteúdo	O design pode ser usado em outros domínios?	0.8

A média ponderada geral é calculada como:

$$\begin{aligned} \text{score_ponderado} &= \frac{\sum(\text{score}_i \times \text{peso}_i)}{\sum(\text{peso}_i)} \\ &= (1.3 \cdot d1 + 1.2 \cdot d2 + 1.3 \cdot d3 + 1.0 \cdot d4 + 1.0 \cdot d5 + 1.0 \cdot d6 + 0.8 \cdot d7) / 7.6 \end{aligned}$$

Relatórios comparativos

O relatório comparativo (RF26) confronta as médias de protótipos `auto` (gerados pelo pipeline) e `manual` (produzidos sem o pipeline) nas 7 dimensões. O relatório está disponível em `/report` na interface web e via `endo-dsl report` na CLI.

Os dados brutos podem ser exportados em CSV para análise estatística:

```
endo-dsl report --csv > avaliacoes.csv
# Formato:
prototype_id,origin,evaluator,bloom_level,domain,d1,d2,...,d7,overall
```


Interpretação dos resultados

O módulo `evaluation/reports.py` gera automaticamente uma interpretação textual com base nos deltas observados. As categorias de interpretação são:

Delta (Δ)	Interpretação
≤ 0.25	Protótipos automáticos essencialmente equivalentes aos manuais
0.25 – 0.50	Diferença pequena; pode não ser estatisticamente significativa
0.50 – 0.75	Diferença moderada; verificar dimensões específicas
> 0.75	Diferença relevante; pipeline necessita ajuste

Atenção metodológica: A endogeneidade (dimensão 3, peso 1.3) é a mais importante para a proposta desta tese. Uma diferença grande nessa dimensão indica que o pipeline gerou mecânicas exógenas — o problema central que a Endo-DSL tenta resolver.

Estudo de Caso: "Comparando Frações"

Um percurso end-to-end completo pela plataforma: da especificação DSL ao protótipo compilado e à avaliação pedagógica.

A especificação DSL

O arquivo `examples/fracoes.endo` é o exemplo canônico da plataforma. Veja a seguir a especificação completa, anotada:

```
// Exemplo: Comparação de Frações – Bloom:
// Analisar
// Compile com: endo-dsl compile examples/
// fracoes.endo

game "Comparando Frações" {

  // — Contexto educacional
  -----

  metadata {
    domain:      "Matemática"
    topic:       "frações"
    bloom:       Analisar           // nível
    dominante do jogo
    age_range:   "10-11"             // alunos do
    5º/6º ano
    duration:    15                   // 15 minutos
  }
```

```

// — Objetivo pedagógico


---



objective obj_comparar {
    description: "Comparar frações com
denominadores diferentes usando
equivalência"
    bloom:      Analisar // verbo
    "comparar" → Analisar ✓
}

// — Mecânica principal: classificação
endógena 

---



mechanic comparacao {
    type:      classification // afim
    a Analisar ✓
    bloom:      Analisar
    addresses:  obj_comparar //
    endereça o objetivo acima
    description: "O jogador classifica
pares de frações em maior, menor ou igual."
    params {
        items:      6
        feedback: immediate
    }
}

// — Mecânica secundária: fixação


---



mechanic revisao {
    type:      quiz
    bloom:
Lembrar // aviso: abaixo do alvo
(Analisar), mas válido
    description: "Rodada de revisão para
fixação dos conceitos."
    params {
        questions: 4
        attempts:  2
    }
}

```

```
// — Loop de jogabilidade

loop principal {
  bloom:      Analisar
  description:
  "Classificar e revisar até consolidar a
  comparação."
  steps {
    comparacao -> revisao
    revisao     -> comparacao
  }
}
}
```

O protótipo HTML5 compilado

Ao compilar com `endo-dsl compile examples/fracoes.endo`, o sistema:

1. Parseia a DSL → AST com título, metadata, 1 objetivo, 2 mecânicas, 1 loop;
2. Valida semanticamente → 0 erros, 1 aviso (mecânica `revisao` abaixo do Bloom alvo);
3. Constrói o content pack com os parâmetros das mecânicas;
4. Renderiza o HTML5 com a mecânica de classificação interativa;
5. Gera o documento de rastreabilidade mapeando `obj_comparar` → `comparacao` → `Analisar`;
6. Persiste no banco e escreve os artefatos em `prototypes/proto_N/`.

O jogo resultante apresenta pares de frações na tela e pede ao jogador para classificar cada par como "maior", "menor" ou "igual". O feedback imediato é fornecido após cada resposta. Após 6 pares classificados, o jogador vai automaticamente para a rodada de revisão (quiz de 4 questões) e retorna ao início do loop.

Este é um jogo **endógeno**: o aluno precisa de fato *comparar frações* para jogar — não é possível avançar sem o conhecimento matemático. A mecânica *é* o aprendizado.

A avaliação pedagógica

Após compilar, acessamos `/evaluate/N` para avaliar o protótipo com o instrumento `endo-eval-1.0`. Uma avaliação típica de um professor do 5º ano:

Dimensão	Nota	Justificativa
Alinhamento pedagógico	5	O jogo realmente exercita comparação de frações
Coerência cognitiva	4	Analisar adequado; revisão em Lembrar como suporte
Endogeneidade	5	Classificar frações é o jogo, não uma barreira
Clareza instrucional	4	Instruções claras; poderiam ser mais visuais
Adequação ao público	4	Adequado para 5º/6º ano; 6 itens é bom
Potencial de engajamento	3	Funcional mas poderia ter narrativa motivacional
Adaptabilidade	5	Fácil de reparametrizar para outros domínios

Média ponderada: $(5 \times 1.3 + 4 \times 1.2 + 5 \times 1.3 + 4 \times 1.0 + 4 \times 1.0 + 3 \times 1.0 + 5 \times 0.8) / 7.6 \approx 4.24$

Esse resultado é representativo dos dados de pesquisa: protótipos automáticos tendem a pontuar bem em endogeneidade (5.0) e alinhamento pedagógico (≥ 4.0) mas abaixo em engajamento e narrativa — áreas que dependem de sensibilidade contextual humana.

Reparametrização: Para adaptar este jogo a Ciências sem recompilar:

```
endo-dsl reparametrize N --domain
"Ciências" --topic "classificação de
seres vivos"
```

O mesmo loop de classificação agora categoriza seres vivos em vez de frações. O alinhamento pedagógico é preservado; apenas o conteúdo muda.

APÊNDICE A

Gramática EBNF Completa

A gramática formal completa da Endo-DSL, em notação EBNF (ISO/IEC 14977).

```
( *
=====
*)
(* Gramática formal da Endo-DSL — RF01,
RF02 *)
(* Notação: EBNF (ISO/IEC
14977).
*)
(*
=====
*)
```

```

spec                = game_decl ;

game_decl           = "game" , string , "{" ,
{ game_member } , "}" ;

game_member         = metadata_block
                    | objective_decl
                    | mechanic_decl
                    | loop_decl
                    | narrative_decl ;

metadata_block      = "metadata" , "{" , {
entry } , "}" ;
entry               = ident , ":" , value ;

objective_decl      = "objective" , ident ,
"{" , { objective_field } , "}" ;
objective_field     = "description" , ":" ,
string
                    | "bloom" , ":" ,
bloom ;

mechanic_decl       = "mechanic" , ident ,
"{" , { mechanic_field } , "}" ;
mechanic_field      = "type" ,
":" , ident
                    | "bloom" ,
":" , bloom
                    | "addresses" ,
":" , ident_list
                    | "description" ,
":" , string
                    | "source_component" ,
":" , string
                    | params_block ;

params_block        = "params" , "{" , {
entry } , "}" ;

loop_decl           = "loop" , ident , "{" , {

```

```

loop_field } , "}" ;
loop_field      = "bloom"      , ":" ,
bloom
                | "description" , ":" ,
string
                | steps_block ;
steps_block     = "steps" , "{" , {
transition } , "}" ;
transition      = ident , "->" , ident ;

narrative_decl = "narrative" , ident ,
"{" , { branch_decl } , "}" ;
branch_decl    = "branch" , ident , "{" ,
{ branch_field } , "}" ;
branch_field   = "bloom"      , ":" ,
bloom
                | "text"      , ":" ,
string
                | "description" , ":" ,
string
                | choice_decl ;
choice_decl    = "choice" , string , "->" , ident ;

ident_list     = ident , { "," , ident } ;
value          = string | number | range
                | bool | ident ;
range          = number , ".." , number ;
bool           = "true" | "false" ;

bloom         = "Lembrar" |
"Compreender" | "Aplicar"
                | "Analisar" |
"Avaliar"      | "Criar" ;

ident          = letter , { letter |
digit | "_" | "." } ;
string         = "'" , { character -
'"' } , "'" ;
number         = [ "-" ] , digit , {
digit } , [ "." , digit , { digit } ] ;

```



```
(* Comentários: "//" até fim de linha; "/"  
* ... */. *)
```

APÊNDICE B

Referência de Comandos CLI

Resumo rápido dos principais comandos. Para a referência completa, consulte `docs/manual-comandos.md` ou execute `endo-dsl --help`.

Comando	Descrição
<code>endo-dsl validate <arquivo></code>	Valida sintaxe e semântica
<code>endo-dsl compile <arquivo></code>	Compila DSL → HTML5
<code>endo-dsl generate -- objective ... --domain ... -- bloom ...</code>	Pipeline automático
<code>endo-dsl new <nome> [-- bloom] [--domain]</code>	Scaffold de arquivo .endo
<code>endo-dsl lint <arquivo></code>	Linting com severidades
<code>endo-dsl fmt <arquivo> [-- write]</code>	Formatação canônica

```
endo-dsl watch <arquivo>
```

Recompila ao
salvar

```
endo-dsl serve [--port  
8000]
```

Interface web

```
endo-dsl library list [--  
bloom] [--type]
```

Lista componentes

```
endo-dsl curate list|  
approve|reject
```

Curadoria

```
endo-dsl reparametrize <id>  
--domain ...
```

Troca domínio

```
endo-dsl report [--csv]
```

Relatório
comparativo

```
endo-dsl stats
```

Métricas de uso

```
endo-dsl doctor
```

Diagnóstico do
ambiente

```
endo-dsl config set <chave>  
<valor>
```

Configuração
persistente

```
endo-dsl theme set <nome>
```

Define tema visual

```
endo-dsl init-db
```

Inicializa banco
SQLite

```
endo-dsl seed-components
```

Popula
componentes
canônicos

```
endo-dsl export [--out  
arquivo.json]
```

Exporta biblioteca

```
endo-dsl import --in  
arquivo.json
```

Importa
componentes

```
endo-dsl grammar
```

Imprime gramática
EBNF

```
endo-dsl limits
```

Imprime limites da
gramática

APÊNDICE C

Glossário

Termo	Definição
AST	Abstract Syntax Tree. Representação estruturada em árvore de uma especificação DSL, gerada pelo parser a partir do texto-fonte.
Backend LLM	O módulo responsável pela geração de texto DSL no pipeline. Dois backends: <code>template</code> (heurístico determinístico) e <code>claude</code> (via API Anthropic).
Bloom	Abreviação da Taxonomia Revisada de Bloom, com seis níveis cognitivos: Lembrar, Compreender, Aplicar, Analisar, Avaliar, Criar.

Canônico

Status de um componente que passou pelo processo de curadoria e foi aprovado pelo curador. Componentes canônicos têm prioridade na recuperação automática.

Compilação

Transformação de uma especificação DSL em um protótipo HTML5 jogável, pelo compilador `compiler/compiler.py`.

Componente pedagógico

Unidade reutilizável de design de jogo na biblioteca. Contém assinatura DSL, metadados de Bloom, parâmetros e referências bibliográficas.

Content pack

Objeto JSON embutido no HTML5 compilado com os dados do jogo (mecânicas, parâmetros, domínio). Permite reparametrização sem recompilar.

DSL

Domain-Specific Language. Linguagem de computação especializada para um domínio específico, com maior expressividade e verificabilidade nesse domínio.

EBNF

Extended Backus-Naur Form. Notação formal para gramáticas de linguagens (ISO/IEC 14977).

	Usada para especificar a gramática da Endo-DSL.
Endogeneidade	Propriedade de um jogo em que o conteúdo educativo é intrinsecamente integrado às mecânicas — aprender é jogar.
Experimental	Status de um componente recém-submetido, aguardando curadoria.
Loop de jogabilidade	Construto DSL que define o fluxo de jogo como grafo de transições entre mecânicas.
LLM	Large Language Model. Modelo de linguagem de grande porte usado no pipeline de geração (ex.: Claude da Anthropic).
Mecânica	Construto DSL que representa um tipo de interação do jogador com o conteúdo. Tipos: quiz, classification, comparison, matching, sequencing, sorting, puzzle, narrative_branch.
Pipeline multi-agente	Sequência de agentes (recuperação RAG → geração LLM → validação → refinamento) que transforma contexto educacional em DSL válida.
Protótipo	Artefato HTML5 gerado pela compilação de uma especificação DSL. Autocontido, jogável offline.

RAG

Retrieval Augmented Generation. Arquitetura que combina recuperação de componentes relevantes com geração de texto por LLM.

Reparametrização

Substituição do conteúdo de domínio em um protótipo já compilado sem recompilar a estrutura.

Rastreabilidade

Documento que mapeia cada objetivo pedagógico às mecânicas que o endereçam e ao nível de Bloom correspondente.

RF

Requisito Funcional. A plataforma possui 26 RFs (RF01–RF26) cobrindo gramática, biblioteca, pipeline, compilação e avaliação.

APÊNDICE D

Referências Bibliográficas

ANDERSON, L. W.; KRATHWOHL, D. R. (Eds.). **A taxonomy for learning, teaching, and assessing**. New York: Longman, 2001.

BLOOM, B. S. et al. **Taxonomy of educational objectives: Handbook I: Cognitive domain**. New York: David McKay, 1956.

CSIKSZENTMIHALYI, M. **Flow: The psychology of optimal experience**. New York: Harper & Row, 1990.

DEURSEN, A. van; KLINT, P.; VISSER, J. Domain-specific languages: An annotated bibliography. **ACM SIGPLAN Notices**, v. 35, n. 6, p. 26–36, 2000.

FRANCE, R.; RUMPE, B. Model-driven development of complex software: A research roadmap. In: **Future of Software Engineering**. IEEE, 2007. p. 37–54.

GEE, J. P. **What video games have to teach us about learning and literacy**. New York: Palgrave Macmillan, 2003.

HABGOOD, M. P. J.; AINSWORTH, S. E. Motivating children to learn effectively: Exploring the value of intrinsic integration in educational games. **Journal of the Learning Sciences**, v. 20, n. 2, p. 169–206, 2011.

HEVNER, A. R. et al. Design science in information systems research. **MIS Quarterly**, v. 28, n. 1, p. 75–105, 2004.

LEWIS, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. **Advances in Neural Information Processing Systems**, v. 33, 2020.

LOPES, R. et al. A framework for evaluating endogenous educational games. **Educational Technology Research**

and Development, v. 71, n. 3, 2023.

MADAAN, A. et al. Self-refine: Iterative refinement with self-feedback. **Advances in Neural Information Processing Systems**, v. 36, 2023.

MALONE, T. W. Toward a theory of intrinsically motivating instruction. **Cognitive Science**, v. 5, n. 4, p. 333–369, 1981.

MERNIK, M.; HEERING, J.; SLOANE, A. M. When and how to develop domain-specific languages. **ACM Computing Surveys**, v. 37, n. 4, p. 316–344, 2005.

PRENSKY, M. **Digital game-based learning**. New York: McGraw-Hill, 2001.

SHINN, N. et al. Reflexion: Language agents with verbal reinforcement learning. **Advances in Neural Information Processing Systems**, v. 36, 2023.

SWELLER, J. Cognitive load during problem solving: Effects on learning. **Cognitive Science**, v. 12, n. 2, p. 257–285, 1988.

TYLER, R. W. **Basic principles of curriculum and instruction**. Chicago: University of Chicago Press, 1949.

VYGOTSKY, L. S. **Mind in society**. Cambridge: Harvard University Press, 1978.

YANNAKAKIS, G. N.; TOGELIUS, J. **Artificial intelligence and games**. Cham: Springer, 2018.

Endo-DSL · Projetando Jogos Educativos com DSL e Taxonomia de Bloom

Versão 1.1.0 · Junho de 2026

PESC/COPPE/UFRJ — Rio de Janeiro, Brasil

Caio Sazeredo · caiosazeredo@cos.ufrj.br

github.com/caiosazeredo/Endo-DSL

Documento gerado em HTML5 autocontido · typeset em Georgia & Helvetica Neue · MIT License